Chamelon: a delta-debugger for OCaml!

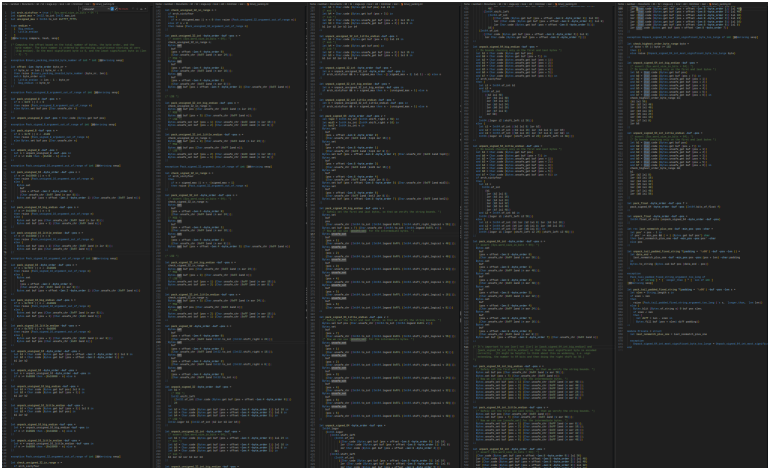
Milla Valnet[1,2,3],
Nathanaëlle Courant[3], Guillaume Bury[3], Pierre Chambart[3], Vincent Laviron[3]
[1]ENS Ulm, [2]Sorbonne Université, [3]OCamlPro

The OCaml language

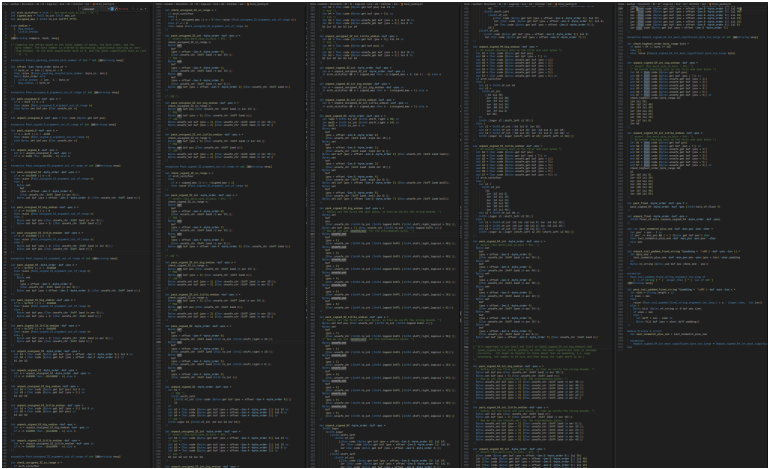



```
type 'a list = Cons of 'a * 'a list | Nil
let rec map f l =
  match l with
  | Cons(h,q) -> Cons(f h, map f q)
  | Nil -> Nil
```

Motivation



Motivation



```
Misc.fatal_error
```

Motivation : for compilers, but not only

- For the developers of the OCaml compiler

Motivation : for compilers, but not only

- For the developers of the OCaml compiler
- To report bugs on confidential code

Motivation : for compilers, but not only

- For the developers of the OCaml compiler
- To report bugs on confidential code
- Adaptable for any other projects on OCaml code!

Structure

- Extracting typed AST

Structure

- Extracting typed AST
- 18 atomical minimization heuristics...

Structure

- Extracting typed AST
- 18 atomical minimization heuristics...
- Each of them applied as much as possible through an iterator...

Structure

- Extracting typed AST
- 18 atomical minimization heuristics...
- Each of them applied as much as possible through an iterator...
- The process repeated until no modification is performed!

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields
- Inline functions

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields
- Inline functions
- Flatten modules

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields
- Inline functions
- Flatten modules
- Suppress dead code

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields
- Inline functions
- Flatten modules
- Suppress dead code
- Supress unused `rec` or arguments

Minimisation heurisitcs: a flavour

- Suppress definitions starting by the end
- Replace expressions by *dummy* values
- Supress constructs/constructs fields/records fields
- Inline functions
- Flatten modules
- Suppress dead code
- Supress unused `rec` or arguments
- etc.

Unit minimization heuristics

Given an integer `i`, a unit heuristics try to minimize the `i`th position of the program which can be simplified.

Unit minimization heuristics

Given an integer `i`, a unit heuristics try to minimize the `i`th position of the program which can be simplified.

Three options :

- ✓ The simplification doesn't remove the error: we minimized the program!

Unit minimization heuristics

Given an integer `i`, a unit heuristics try to minimize the `i`th position of the program which can be simplified.

Three options :

- ✓ The simplification doesn't remove the error: we minimized the program!
- ✗ The simplification removes the error: we don't want to apply it.

Unit minimization heuristics

Given an integer `i`, a unit heuristics try to minimize the `i`th position of the program which can be simplified.

Three options :

- ✓ The simplification doesn't remove the error: we minimized the program!
- ✗ The simplification removes the error: we don't want to apply it.
- STOP The position is greater than the last modifiable position.

Algebraic return type

```
type 'a minimized_step_result =
  | New_state of 'a
  (* New (smaller) state that produces an error *)
  | Change_removes_error
  (* This change removes the error, but other might be possible *)
  | No_more_changes
  (* The last possible position for changes has been reached *)
```

The main loop

```
let minimize_basic (state : 'a)
  (f : 'a -> pos:int -> 'a minimized_step_result) : 'a * bool =
  let rec aux (state : 'a) (pos : int) (ever_changed : bool) =
    match f state ~pos with
    | New_state nstate -> aux nstate pos true
    | Change_removes_error -> aux state (pos + 1) ever_changed
    | No_more_changes -> (state, ever_changed)
  in
  aux state 0 false
```

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=0

The main loop



pos=1

The main loop



pos=1

The main loop



pos=1

The main loop



pos=1

The main loop



pos=1

The main loop



pos=1

The main loop



pos=2

The main loop



pos=2

The main loop



pos=2

The main loop



pos=2

The main loop



pos=2

The main loop



pos=2

The main loop



pos=2

The main loop



pos=3

The main loop



pos=3

The main loop



pos=4

The main loop



pos=4

The main loop



pos=4

The main loop



pos=4

Concretely

- 18 minimization heuristics

Concretely

- 18 minimization heuristics
- Representing 3000 LoC

Concretely

- 18 minimization heuristics
- Representing 3000 LoC
- Declined to ocamlc and flambda2 compilers

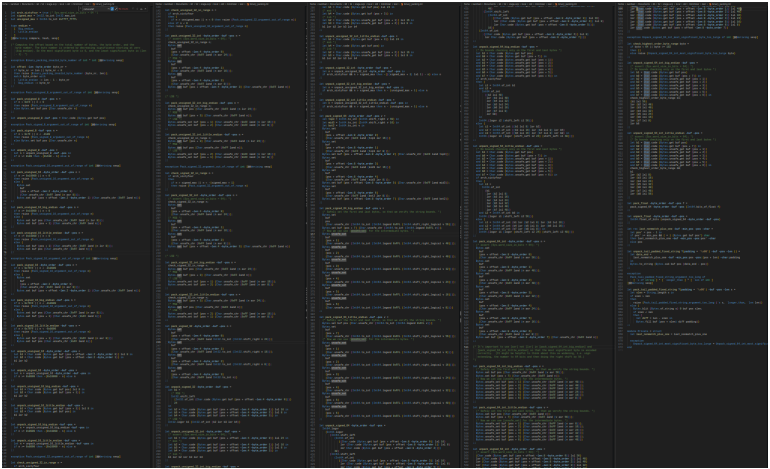
Concretely

- 18 minimization heuristics
- Representing 3000 LoC
- Declined to ocamlc and flambda2 compilers
- Declinable at low cost to different AST version via a compatibility library

Concretely

- 18 minimization heuristics
- Representing 3000 LoC
- Declined to ocamlc and flambda2 compilers
- Declinable at low cost to different AST version via a compatibility library
- Giving concrete results on real programs!

Demonstration



```
Misc.fatal_error
```

Demonstration

test > binary_packing_min.ml

```
1  let offset ~byte_order  byte_nr =
2    match byte_order with | `Little_endian -> 0 | `Big_endian -> byte_nr
3
4  let pack_unsigned_16 ~byte_order  =
5    __ignore__ ((offset ) ~byte_order 0);
6    __ignore__ ((__dummy2__ ()) ((offset ) ~byte_order 1));
7    __dummy2__ ()
```

```
Misc.fatal_error
```

Chamelon: turn your code into a baby camel!



A free and open-source tool!

```
>>$ git clone https://github.com/Ekdohibs/chamelon
>>$ dune build #with OCaml 4.14
```

A free and open-source tool!

```
>>$ git clone https://github.com/Ekdohibs/chamelon
>>$ dune build #with OCaml 4.14
```

For examples, see the artifact at: `https://zenodo.org/records/12520654`

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

The tool will display:

- `Starting to minimize minimized_input.ml`

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

The tool will display:

- `Starting to minimize minimized_input.ml`
- `Trying heuristic-name:  pos=i...` for each atomic steps.

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

The tool will display:

- `Starting to minimize minimized_input.ml`
- `Trying heuristic-name: pos=i...` for each atomic steps.
  - ✓ `Reduced.` when the transformation is applied.

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

The tool will display:

- `Starting to minimize minimized_input.ml`
- `Trying heuristic-name: pos=i...` for each atomic steps.
  - ✓ `Reduced.` when the transformation is applied.
  - ✗ `Removes error.` when the transformation removes the error.

chamelon command

```
chamelon -c "[command]" -e "[error]" input.ml
```

The tool will display:

- `Starting to minimize minimized_input.ml`
- `Trying heuristic-name: pos=i...` for each atomic steps.
  - ✓ `Reduced.` when the transformation is applied.
  - ✗ `Removes error.` when the transformation removes the error.
  - `No more changes.` when the end of the program is reached.

From one file to a full project: supporting multifile

From one file to a full project: supporting multifile

- Suppress as much files as possible

From one file to a full project: supporting multifile

- Suppress as much files as possible
- Merge as much files as possible

From one file to a full project: supporting multifile

- Suppress as much files as possible
- Merge as much files as possible
- Adapt heuristics to propagate definition changes

From one file to a full project: supporting multifile

- Suppress as much files as possible
- Merge as much files as possible
- Adapt heuristics to propagate definition changes

We feed the minimizer with the file lists - `ocamldep` gives us the order of compilation.

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}8$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}8$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}=0 \wedge \texttt{lgth}=2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}0 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



pos=1 ∧ lgth=1

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}4$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}2$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}1 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}2 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}2 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}2 \wedge \texttt{lgth}{=}1$

Optimisation

We replace our linear iteration method by a dichotomy:



$\texttt{pos}{=}2 \wedge \texttt{lgth}{=}1$

We divide minimization time by 10 for a 4000-lines!

Future works

- Make Chamelon compatible with a dune project

Future works

- Make Chamelon compatible with a dune project
- More tests, on more files, to improve precision, robustness, efficiency, etc.

Results

- A delta-debugger to assist the development of your project manipulating OCaml code...

Results

- A delta-debugger to assist the development of your project manipulating OCaml code...
- Minimizing efficiently real example...

Results

- A delta-debugger to assist the development of your project manipulating OCaml code...
- Minimizing efficiently real example...
- Saving time for its users!

Thank you for your attention!